

Použití SQL v RPG

Vladimír Župka, 2025

Předkompilátor SQL pro jazyk RPG

Zdrojový typ **SQLRPGLE**

Kompilace **CRTSQLRPGI** – Create SQL ILE RPG Object

Volba **14** v PDM dosadí `OBJTYPE(*PGM)`

Volba **15** v PDM dosadí `OBJTYPE(*MODULE)`

`OBJTYPE(*SRVPGM)` nemá volbu v PDM

Source member

```
CRTSQLRPGI OBJ(STAOBR_SQL) SRCFILE(QRPGLESRC) SRCMBR(STAOBR_SQL)  
                OBJTYPE(*PGM)  OUTPUT(*PRINT) DBGVIEW(*SOURCE)
```

Stream file

```
CRTSQLRPGI OBJ(VZUPKA/STAOBR_SQL)  
                SRCSTMF( '/home/VZUPKA/STAOBR_SQL.SQLRPGLE' )  
                OBJTYPE(*PGM)  OUTPUT(*PRINT) DBGVIEW(*SOURCE)
```

Parametry překlada

COMMIT (<u>*CHG</u>)	*NONE *ALL ...
OBJTYPE (<u>*PGM</u>)	*MODULE *SRVPGM
OPTION (<u>*SYS</u> *SQL	jmenná konvence
<u>*JOB</u> *SYSVAL *PERIOD *COMMA	
...)	
CLOSQLCSR (<u>*ENDACTGRP</u>)	*ENDMOD kdy se uzavře kurzor
DFTRDBCOL (<u>*NONE</u>)	předvolené SQL schema (default collection)
DYNDFTCOL (<u>*NO</u>)	*YES – DFTRDBCOL také pro dynamické příkazy
TOSRCFILE (<u>QTEMP/QSQLTEMP1</u>)	kam je uložen předkompilovaný zdrojový text
DBGVIEW (<u>*NONE</u>)	*SOURCE, *STMT, *LIST
...	

Zápis příkazů v RPG programu

Ve volném formátu

EXEC SQL *příkaz* ;

V pevném formátu

C/EXEC SQL *příkaz*

C/END-EXEC

Alternativní a doplňkové volby pro SQL příkazy. Některé jsou shodné s parametry předkompilátoru.

Exec SQL **SET OPTION** LANGID = CSY, SRTSEQ = *LANGIDSHR,
 DATFMT = *ISO, COMMIT = *NONE ;

COMMIT = *CHG *NONE *ALL ...
NAMING = *SYS *SQL
DATFMT = *JOB *ISO ...
DATSEP = *JOB *PERIOD *COMMA *DASH *BLANK
DFTRDBCOL = *NONE
LANGID = *JOB *JOBRUN CSY ENU DEU ...
SRTSEQ = *JOB *HEX *JOBRUN *LANGIDUNQ *LANGIDSHR
TIMFMT = *HMS *ISO *EUR *USA *JIS
TIMSEP = *JOB *COLON *PERIOD *COMMA *BLANK
...

Soubory – tabulky

Soubor CENIKP – Ceník materiálu

A			UNIQUE
A	R CENIKPF0		
A	<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A	CENAJ	10P 2	COLHDG('Cena/j.')
A	NAZEV	30	COLHDG('Název zboží')
A	K MATER		

Soubor STAVYP – Stavby materiálových zásob

A			UNIQUE
A	R STAVYPF0		
A	ZAVOD	2	COLHDG('Záv')
A	SKLAD	2	COLHDG('Skl')
A	<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A	MNOZ	10P 2	COLHDG('Množství')
A	K ZAVOD		
A	K SKLAD		
A	K MATER		

Soubor OBRATP – Obraty materiálu

A	R OBRATPF0		
A	ZAVOD	2	COLHDG('Záv')
A	SKLAD	2	COLHDG('Skl')
A	<u>MATER</u>	5	COLHDG('Číslo' 'mater.')
A	MNOBR	10P 2	COLHDG('Množství obratu')
A	K ZAVOD		
A	K SKLAD		
A	K MATER		

Statický SELECT

Program STASEL

```
**free
```

```
Dcl-DS ceny ExtName('*LIBL/CENIKP') End-DS; // host variables
```

```
Exec SQL declare CUR cursor for  
      select * from CENIKP  
      where CENAJ between 2.5 and 300  
      order by CENAJ asc for read only;
```

```
Exec SQL open CUR ;
```

```
DoW *On;  
  Exec SQL fetch CUR into :ceny;  
  If sqlcode > 0;  
    leave;  
  EndIf;  
  Snd-Msg *info MATER + ' ' + %editc(CENAJ: 'K') + ' ' + NAZEV;  
EndDo;
```

```
Exec SQL close CUR;
```

```
Return;
```

Chybové stavy

Viz dokumentaci: <https://www.ibm.com/docs/en/i/7.5.0?topic=codes-listing-sqlstate-values>.

SQLSTATE obecnější		SQLCODE IBM
00000	Operace byla úspěšná , bez varování nebo výjimky.	0
01503	Počet výsledných sloupců je větší než poskytnutý počet proměnných.	+000, +030
02000	Nastala jedna z výjimek: - Výsledek příkazu SELECT INTO nebo subselektu v příkazu INSERT je prázdná tabulka. - Počet řádků určených v hledacím příkazu UPDATE nebo DELETE je nula. - Pozice kurzoru v příkazu FETCH je za posledním řádkem výsledné tabulky (konec dat jako EOF). - Orientace příkazu FETCH je nesprávná.	100
07001	Počet proměnných neodpovídá počtu parametrů (markerů)	-313
09000	Trigger v SQL příkazu selhal.	-723
42703	Zjištěn nedefinovaný sloupec nebo jméno parametru.	-205, -206, -213, -5001

Dynamický SELECT bez parametrů

Program DYNSEL

```
**free
```

```
Dcl-DS *N  ExtName('*LIBL/CENIKP')  End-DS;
```

```
Dcl-S spodni      packed(5: 2) inz( 2.5 );
```

```
Dcl-S horni      packed(5: 2) inz( 300 );
```

```
Dcl-S sel_stmt  char( 500 ) inz;
```

```
sel_stmt = 'select MATER, CENAJ, NAZEV from CENIKP +  
           where CENAJ between ' + %char(spodni) + ' and ' + %char(horni) +  
           ' order by CENAJ asc for read only';
```

```
Exec SQL prepare PREP from :sel_stmt ;
```

```
Exec SQL declare CUR cursor for PREP;
```

```
Exec SQL open CUR ;
```

```
Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;
```

```
DoW sqlstate < '02000';
```

```
    snd-msg *info MATER + ' ' + %editc(CENAJ: 'K') + ' ' + NAZEV;
```

```
    Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;
```

```
EndDo;
```

```
Exec SQL close CUR;
```

```
Return;
```


Dynamický SELECT s markery

Program DYNSEL_MK

```
**free
```

```
Dcl-S sel_stmt      char(500) inz;  
Dcl-S spodni       packed(5: 2) inz( 2.5 );  
Dcl-S horni        packed(5: 2) inz( 300 );  
Dcl-DS *N  ExtName('CENIKP')  End-DS;
```

```
sel_stmt = 'select MATER, CENAJ, NAZEV from CENIKP +  
           where CENAJ between ? and ? +  
           order by CENAJ asc for read only' ;
```

```
Exec SQL prepare PREP from :sel_stmt ;  
Exec SQL declare CUR cursor for PREP;  
Exec SQL open CUR using :spodni, :horni;
```

```
Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;  
DoW sqlcode = 0;  
    Snd-Msg *info MATER + ' ' + %editc(CENAJ: 'K') + ' ' + NAZEV;  
    Exec SQL fetch CUR into :MATER, :CENAJ, :NAZEV;  
EndDo;
```

```
Exec SQL close CUR;  
Return;
```

Stavy a obraty – příkazy SELECT, UPDATE

Program STAOB3_SQL

```
**free
// Oprava množství materiálů přičtením množství obrátů
Exec SQL set option COMMIT = *NONE;

Dcl-DS stavy ExtName('*LIBL/STAVYP') End-DS; // host variables
Dcl-DS obraty ExtName('*LIBL/OBRATP') qualified End-DS; // odstraní duplicitu

Exec SQL declare CS cursor for
        select ZAVOD, SKLAD, MATER, MNOZ from STAVYP;
Exec SQL open CS;

Exec SQL fetch from CS into :stavy;
DoW sqlstate < '02000';
    Exec SQL update STAVYP set MNOZ = MNOZ +
        ( select sum(MNOBR) from OBRATP
          where ZAVOD = :ZAVOD
            and SKLAD = :SKLAD
            and MATER = :MATER
          group by ZAVOD, SKLAD, MATER
        )
    where ZAVOD = :ZAVOD and SKLAD = :SKLAD and MATER = :MATER;
    Exec SQL fetch from CS into :stavy;
EndDo;

Exec SQL close CS;
Return;
```

Stavy a obraty – příkaz MERGE

Program STAOBM_SQL

****free**

Exec SQL set option COMMIT = *NONE;

Exec SQL **MERGE INTO STAVYP S** // statický příkaz
 USING (**select** O.ZAVOD, O.SKLAD, O.MATER, sum(O.MNOBR) *SUMA_OBRATU*
 from **OBRATP** O
 group by O.ZAVOD, O.SKLAD, O.MATER
) as OBR
 ON (S.ZAVOD = OBR.ZAVOD
 and S.SKLAD = OBR.SKLAD
 and S.MATER = OBR.MATER)
 when **MATCHED** **then**
 update set MNOZ = MNOZ + OBR.*SUMA_OBRATU*
-- **when** **NOT** **MATCHED** **then**
-- **insert** (ZAVOD, SKLAD, MATER, MNOZ)
-- **values**(OBR.ZAVOD, OBR.SKLAD, OBR.MATER, OBR.*SUMA_OBRATU*)
;

Return;

Stavy a obraty – RPG

Program STAOBR

```
**free

dcl-f STAVYP keyed usage(*update);
dcl-f OBRATP keyed;

read OBRATP;
do not %eof(OBRATP);
  chain (ZAVOD: SKLAD: MATER) STAVYP;
  if %found;
    MNOZ += MNOBR;
    update STAVYPF0 %fields(MNOZ);
  endif;
  read OBRATP;
enddo;

*inlr = *on;
```

Dynamický příkaz – EXECUTE IMMEDIATE

Program DYNEX_IM

```
**free
```

```
Dcl-S DYNST          Char(500) Inz;  
Dcl-S Limit          Packed(10: 2) Inz(500.00);
```

```
Exec SQL set option COMMIT = *NONE;
```

```
DYNST = 'update CENIKP set CENAJ = CENAJ * 1.10 +  
          where CENAJ < ';
```

```
DYNST = %trim(DYNST) + ' ' + %char(Limit) ;
```

```
Exec SQL execute immediate :DYNST;
```

```
return;
```

Dynamický příkaz – PREPARE, EXECUTE, marker

Program DYNEX_MK

```
**free
Dcl-S DYNST          Char(500) Inz;

Dcl-PI *n;
    Limit Packed(10: 2); // parametr z CMD příkazu
End-PI;

Exec SQL set option COMMIT = *NONE;

DYNST = 'update CENIKP set CENAJ = CENAJ * 1.10 +
        where CENAJ < ? ';
Exec SQL prepare STMT from :DYNST;
Exec SQL execute STMT using :Limit;

return;
```

CL příkaz k vyvolání programu:

```
CMD          PROMPT('Volání SQL programu DYNEX_MK')
PARM         KWD(LIMIT) TYPE(*DEC) LEN(10 2) +
             DFT(250) PROMPT('Limit ceny:')
```

```
DYNEX_MK    LIMIT(250)
```

Statické a dynamické příkazy

Statické příkazy jsou zapsány přímo v příkazu EXEC SQL

Dynamické příkazy jsou zapsány v textové proměnné

Postup příkazů pro SELECT

```
DECLARE SCROLL kurzor CURSOR FOR SELECT ...  
OPEN kurzor  
FETCH FIRST FROM kurzor INTO :proměnná, ...  
      (FIRST, NEXT, LAST, PRIOR, BEFORE, AFTER, CURRENT, RELATIVE)  
CLOSE kurzor
```

Postup příkazů pro dynamický SELECT bez parametrů (markerů)

```
text-příkazu = 'SELECT ... '  
PREPARE připravený-příkaz FROM :text-příkazu  
DECLARE SCROLL kurzor CURSOR FOR připravený-příkaz  
... jako výše
```

Postup příkazů pro dynamický SELECT s parametry (markery)

```
text-příkazu = 'SELECT ... WHERE xyz BETWEEN ? AND ? ... '  
PREPARE připravený-příkaz FROM :text-příkazu  
DECLARE SCROLL kurzor CURSOR FOR připravený-příkaz  
OPEN kurzor USING :proměnná1, :proměnná2  
... jako výše
```

Postup příkazů pro ostatní dynamické příkazy bez parametrů (markerů)

```
text-příkazu = 'UPDATE ... SET ... '  
EXECUTE IMMEDIATE :text-příkazu
```

Postup příkazů pro ostatní dynamické příkazy s parametry (markery)

```
text-příkazu = 'UPDATE ... SET ... WHERE xyz BETWEEN ? AND ?'  
PREPARE připravený-příkaz FROM :text-příkazu  
EXECUTE připravený-příkaz USING :proměnná1, :proměnná2
```


Lokální SQL tabulka v podproceduře

Program LOC_SQL

```
.....
**free
  // hlavní procedura volá podproceduru
ctl-opt dftactgrp(*no); // kvůli podproceduře v modulu

  // volání podprocedury
dsply %editc(VRATIT_CENU('00001'): 'P'); // volání podprocedury
return;

.....
  // podprocedura vrací cenu pro číslo materiálu
dcl-proc VRATIT_CENU export;

  dcl-DS cenik_ds extname('CENIKP') qualified end-DS;
  dcl-PI *N packed(10: 2); // rozhraní podprocedury
    material like(cenik_ds.MATER) CONST; // nebo VALUE
  end-PI;

  Exec SQL select CENAJ into :cenik_ds.CENAJ from CENIKP
        where MATER = :material;
  if sqlstate >= '02000';
    cenik_ds.CENAJ = -1;
  endif;
  return cenik_ds.CENAJ;
end-proc;
.....
```

Lokální soubor v podproceduře

Program LOCFILE

```
.....
**free
  // hlavní program volá podproceduru
ctl-opt dftactgrp(*no);

  // volání podprocedury
dsply %editc( vratit_cenu('00001'): 'P');
return;

.....
  // podprocedura vrací cenu pro číslo materiálu
dcl-proc VRATIT_CENU export;
  dcl-F CENIKP keyed; // příp. STATIC - nechá soubor otevřený
  dcl-DS cenik_ds likerec(CENIKPF0); // je nutná datová struktura

  dcl-PI *N packed(10: 2); // rozhraní podprocedury
    material like(cenik_ds.MATER) CONST; // nebo VALUE
  end-PI;

  chain(e) material CENIKP cenik_ds; // čte do datové struktury
  if not %found();
    cenik_ds.CENAJ = -1;
  endif;
  return cenik_ds.CENAJ;
end-proc;
.....
```

Dlouhá jména

Vytvoření SQL tabulky a naplnění záznamy

Program DL_JM1

```
**free
```

```
Exec SQL set option COMMIT = *NONE ;
```

```
Exec SQL CREATE OR REPLACE TABLE CENY_ZBOZI  
      ( CISLO_ZBOZI      CHAR(5)          UNIQUE,  
        CENA_ZA_JEDNOTKU DEC(12, 2),  
        NAZEV_ZBOZI      CHAR(50) CCSID 870  
      ) ;
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00003', 1.25, 'Prádelní šňůra');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00004', 10.50, 'Ponožky pánské tmavé');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00005', 120.00, 'Tričko bílé');
```

```
Exec SQL INSERT INTO CENY_ZBOZI values ('00006', 10.55, 'Ponožky pánské bílé');
```

```
...
```

```
return;
```

V protokolu o kompilaci najdeme odpovídající systémová jména

Prvních 5 znaků zůstává, přidá se pořadové číslo.

```
...  
  
CENA_ZA_JEDNOTKU      ****      COLUMN  
                        8  
CENA_ZA_JEDNOTKU      6          COLUMN FOR CENA_00001 IN CENY_ZBOZI  
  
CENA_00001            6          DECIMAL(12,2) COLUMN IN CENY_ZBOZI  
  
...
```

Alternativně si můžeme volit vlastní systémová jména

```
Exec SQL CREATE OR REPLACE TABLE CENY_ZBOZI FOR SYSTEM NAME kratší-jméno  
      ( CISLO_ZBOZI FOR COLUMN SYSTEM NAME kratší-jméno CHAR(5),  
        CENA_ZA_JEDNOTKU FOR COLUMN SYSTEM NAME kratší-jméno DEC(12, 2),  
        NAZEV_ZBOZI FOR COLUMN SYSTEM NAME kratší-jméno CHAR(50)  
      );
```

Výpis cen zboží

Program DL_JM2

```
**free
```

```
Dcl-DS *N ExtName('CENY_ZBOZI') End-DS; // host variables
```

```
Exec SQL declare CS cursor for
      select CISLO_ZBOZI, CENA_ZA_JEDNOTKU, NAZEV_ZBOZI
      from   CENY_ZBOZI
      order by CISLO_ZBOZI for read only;
```

```
Exec SQL open CS;
```

```
Exec SQL fetch from CS into :CISLO00001, :CENA_00001, :NAZEV00001 ;
dow sqlstate < '02000';
      snd-msg CISLO00001 + ' ' + %char(CENA_00001) + NAZEV00001;
      Exec SQL fetch from CS into :CISLO00001, :CENA_00001, :NAZEV00001 ;
enddo;
```

```
Exec SQL close CS;
```

```
return;
```

Subfile v RPG s SQL

Obrazkový soubor STAVYW2

```
A*N01N02N03T.Name+++++RLen++TDpBLinPosFunctions+++++
A                                     DSPSIZ(*DS3)
A                                     REF(REFMZP)
A                                     CA03(03 'Konec')
*   Datový formát podsouboru
A           R DATA                                     SFL
A           ZAVOD      R      O  7  3
A           SKLAD      R      O  7 10
A           MATER      R      O  7 17
A           MNOZ      R      O  7 30EDTCDE(K)
*   Řídicí formát podsouboru - záhlaví
A           R RIDICI                                     SFLCTL(DATA)
A                                     SFLSIZ(0100)
A                                     SFLPAG(0005)
A 51                                     SFLDSP
A                                     SFLDSPCTL
A 51                                     SFLEND
A                                     2 18'Přehled stavů'
A                                     4  3'Závod'
A                                     4 10'Sklad'
A                                     4 17'Materiál'
A                                     4 35'Množství'
```

Plnění podsouboru pomocí SQL

Program STAPSZOB2

```
**free
dcl-f STAVYW2 workstn(*ext) usage(*input: *output) sfile(DATA: idx);
dcl-ds STAVYP extname('STAVYP') end-ds; // host variables
dcl-s idx int(10);

// získám data se souboru STAVYP
Exec SQL declare CURSOR cursor for
    select ZAVOD, SKLAD, MATER, MNOZ from STAVYP
    order by SKLAD, MATER asc for read only;
Exec SQL open CURSOR;

// naplním podsoubor z výsledku dotazu pomocí host variables
dow sqlstate < '02000';
    Exec SQL fetch from CURSOR into :ZAVOD, :SKLAD, :MATER, :MNOZ ;
    idx += 1;
    write DATA; // zapíše nový záznam do podsouboru
enddo;
Exec SQL close CURSOR;

*in51 = *on;
exfmt RIDICI;

if *in03;
    *inlr = *on; // uvolním obraz. soubor
    return;
endif;
```

Plnění podsouboru bez SQL – datové struktury pro I/O

Program STAPSZOBDS

```
**free
dcl-f STAVYW2 workstn(*ext) usage(*input: *output)
           sfile(DATA: idx);
dcl-f STAVYP keyed; // disk(*ext) usage(*input)
dcl-ds READ_DS extname('STAVYP': *input) end-ds;
dcl-ds RID_DS likerec(RIDICI: *all);
dcl-ds DAT_DS likerec(DATA: *all);
dcl-s idx int(10);

read STAVYP read_ds;
dow not %eof;
    idx += 1;
    eval-corr DAT_DS = read_ds; // EVAL-CORR - přesun podle jmen podpolí
    write DATA DAT_DS;
    read STAVYP read_ds;
enddo;

RID_DS.in51 = *on;
exfmt RIDICI RID_DS; // EXFMT s datovou strukturou

if RID_DS.in03;
    *inlr = *on;
    return;
endif;
```


Datové typy RPG a SQL

Viz stránku <https://www.ibm.com/docs/en/i/7.5.0?topic=uhviiratus-declaring-host-variables-in-ile-rpg-applications-that-use-sql>.

RPG	SQL
CHAR(n)	CHAR(n)
VARCHAR(n:2)	VARCHAR(n)
UCS2(n)	GRAPHIC(n)
VARUCS2(n:2)	VARGRAPHIC(n)
PACKED(n:m)	DECIMAL(n, m)
ZONED(n:m)	NUMERIC(n, m)
INT(5)	SMALLINT
INT(10)	INTEGER
INT(20)	BIGINT
BINDEC(1-4:0)	SMALLINT
BINDEC(5-9:0)	INTEGER
FLOAT(4)	FLOAT(24) REAL
FLOAT(8)	FLOAT(53) FLOAT
IND	BOOLEAN
DATE(*DMY-)	DATE
TIME(*HMS.)	TIME
TIMESTAMP(n)	TIMESTAMP(n)

Neodpovídající typy

```
dcl-S bin_1      SQLTYPE(BINARY: 50);      // CHAR(50) CCSID(*HEX);
dcl-S varbin_1   SQLTYPE(VARBINARY: 100);   // VARCHAR(100) CCSID(*HEX);

dcl-S FIELD_1    SQLTYPE (CLOB: 1000 );     // DCL-DS FIELD_1;
                                           //      FIELD_1_LEN   UNS(10);
                                           //      FIELD_1_DATA CHAR(1000);
                                           // END-DS FIELD_1;

Exec SQL set :FIELD_1 = 'ABCD';             // příkaz SQL v RPG
FIELD_1_DATA = 'ABCD';                      // příkaz v RPG
```

Externí SQL rutiny

Externí rutiny, tj. trigger, procedura, funkce jsou obšírně vysvětleny v dokumentu <https://www.redbooks.ibm.com/redbooks/pdfs/sg246503.pdf>

Trigger externí – zvýšení ceny nejvýše o 10% při aktualizaci záznamu

Přidání k tabulce - registrace

```
ADDPFTRG FILE(CENIKP) TRGTIME(*BEFORE) TRGEVENT(*UPDATE) PGM(TG_CENA_U)  
        RPLTRG(*YES) TRG(TG_CENA_U) TRGLIB(*FILE) ALWREPCHG(*YES)
```

Zjištění triggerů způsobujících změnu

```
select trigger_schema, trigger_name  from QSYS2/SYSTRIGGER  
       where ALLOW_REPEATED_CHANGE = 'YES'  
       and trigger_schema not like 'Q%'
```

Odstranění

```
RMVPFTRG FILE(CENIKP) TRG(TG_CENA_U) TRGLIB(*FILE)  
DROP TRIGGER TG_CENA_U
```

Změna

```
CHGPFTRG FILE(VZSQL/OBJDET_T) TRG(TRG_MNOBJU) TRGLIB(VZSQL) STATE(*DISABLED)  
CHGPFTRG FILE(VZSQL/OBJDET_T) TRG(TRG_MNOBJU) TRGLIB(VZSQL) STATE(*ENABLED)
```

Trigger externí – RPG program

Program TG_CENA_U

```
**free
Dcl-PI *n;      // vstupní parametry z databázového systému
    Buf         LikeDS(TrgBuffer);
    Len         Uns(10); // nepotřebuji
End-PI;

    // Trigger buffer (1. parametr) – obsahuje data obou záznamů
Dcl-DS TrgBuffer          Len(1000);
    // Statická oblast
    FileName              Char(10);
    LibraryName            Char(10);
    MemberName             Char(10);
    TrgEvent               Char(1);
    TrgTime                Char(1);
    CmtLckLvl              Char(1);
    // Dynamická oblast
    OldRecOffset          Uns(10) pos(49);
    OldRecLen              Uns(10);
    NewRecOffset          Uns(10) pos(65);
    NewRecLen              Uns(10);
End-DS;
```

```

//    Starý záznam (before update)
Dcl-DS OldRecord  ExtName('CENIKP') Based(OldRecPtr) Qualified End-DS;
//    Nový záznam (before insertion)
Dcl-DS NewRecord  ExtName('CENIKP') Based(NewRecPtr) Qualified End-DS;

Dcl-S OldRecPtr          Pointer; // Prázdný ukazatel na Starý záznam
Dcl-S NewRecPtr          Pointer; // Prázdný ukazatel na Nový záznam

OldRecPtr = %Addr(Buf) + Buf.OldRecOffset; // Adresa Starého záznamu
NewRecPtr = %Addr(Buf) + Buf.NewRecOffset;  // Adresa Nového záznamu

If (NewRecord.CENAJ > 100 and NewRecord.CENAJ > OldRecord.CENAJ * 1.10) ;
    NewRecord.CENAJ = OldRecord.CENAJ * 1.10 ;
EndIf;

Return;

```

Uložená procedura externí – hodnota všeho materiálu ve skladě

Skript VZSQLPGM/PR_CEN_E

Definiční skript

```
SET SCHEMA = VZUPKA;
CREATE OR REPLACE PROCEDURE CENA_MAT
    ( in CISLO_ZAVODU    CHAR(2),          -- číslo závodu
      in CISLO_SKLADU    CHAR(2),          -- číslo skladu
      out CASTKA_CELKEM decimal (11, 2)    -- součet částek cena krát množství
    )
language RPGLE                                -- jazyk ILE RPG
parameter style general                        -- nepředávají se null-indikátory
not deterministic
reads SQL data
program type MAIN                            -- procedura je hlavní program
external name PR_CEN_R                       -- realizuje uloženou proceduru
```

Umístíme skript do zdrojového členu PR_CEN_E a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(PR_CEN_E) MARGINS(160)
```

nebo

V aplikaci **ACS** (IBM i Access Client Solutions) vložíme skript do okna **Run SQL Scripts** a spustíme.

Procedura externí – RPG program (s SQL)

Program VZSQLPGM/PR_CEN_R

****free**

// Parametry uložené procedury

Dcl-PI *n;

 Zavod Char(2); // in

 Sklad Char(2); // in

 Suma Packed(11:2); // out

End-PI;

```
Exec SQL select sum(CENAJ * MNOZ) into :Suma      -- out
      from STAVYP as S
      join CENIKP as C on C.MATER = S.MATER
      where ZAVOD = :Zavod and SKLAD = :Sklad ;
```

Return;

Volání procedury v RPG programu

Program VZSQLPGM/PR_CEN_C

```
Dcl-F QPRINT      Printer(120);
```

```
Dcl-PI *n; // vstupní parametry uložené procedury
```

```
    Zavod          Char(2);
```

```
    Sklad          Char(2);
```

```
End-PI;
```

```
Dcl-S Suma          Packed(11:2); // výstupní parametr
```

```
Exec SQL  CALL  CENA_MAT ( :Zavod, :Sklad, :Suma );
```

```
Except DETAIL; // tisk výsledků
```

```
*inlr = *on; // uzavře soubor QPRINT
```

```
QQPRINT      E          DETAIL          1
O
                                'Celková cena materiálu '
QQPRINT      E          DETAIL          1
O
                                'Závod: '
O          Zavod          +1
QQPRINT      E          DETAIL          1
O
                                'Sklad: '
O          Sklad          +1
QQPRINT      E          DETAIL          1
O
                                'Celkem: '
O          Suma          P          +1
```


CL příkaz k vyvolání programu:

```
CMD      PROMPT('Volání SQL programu PR_CEN_C')
PARM     KWD(ZAVOD) TYPE(*CHAR) LEN(2) +
          DFT('01') +
          PROMPT('Číslo závodu:')
PARM     KWD(SKlad) TYPE(*CHAR) LEN(2) +
          DFT('01') +
          PROMPT('Číslo skladu:')
```

```
PR_CEN_C ZAVOD('01') SKLAD('01')
```

Výsledný tisk

```
Celková cena materiálu
Závod:   01
Sklad:   01
Celkem:      60158.16
```

User defined function (UDF) externí – vrací jednotkovou cenu

Skript VZUPKA/FU_CEN_E

Definiční skript

```
SET SCHEMA = VZUPKA;  
CREATE OR REPLACE FUNCTION VZUPKA/VRAT_CENU_MATERIALU (MATERIAL CHAR(5))  
  RETURNS DECIMAL (10, 2)  
  LANGUAGE RPGLE  
  PARAMETER STYLE GENERAL  
  NOT DETERMINISTIC NO SQL  
  PROGRAM TYPE SUB          -- funkci realizuje podprocedura  
  NOT FENCED                -- není vázána na stejnou úlohu  
  NO FINAL CALL             -- není první a poslední volání  
  EXTERNAL NAME VZUPKA.FU_CEN_R(FU_CEN_R) -- sevisní program a podprocedura
```

Umístíme skript do zdrojového členu FU_CEN_E a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(FU_CEN_E) MARGINS(160)
```

Funkce externí – RPG podprocedura

Program VZUPKA/FU_CEN_R

```
**free
ctl-opt nomain;
dcl-PROC FU_CEN_R export;
    dcl-F CENIKP keyed; // příp. STATIC - nechá soubor otevřený
    dcl-DS cenik_ds likerec(CENIKPF0); // je nutná datová struktura

    dcl-PI *N packed(10: 2); // rozhraní podprocedury
        material like(cenik_ds.MATER) CONST; // nebo VALUE
    end-PI;

    chain(e) material CENIKP cenik_ds; // čte do datové struktury
    if not %found();
        cenik_ds.CENAJ = -1;
    endif;
    return cenik_ds.CENAJ;
end-PROC FU_CEN_R;
```

- Lokální soubor negeneruje popisy I a O s proměnnými.
- Pro datová pole je nutné použít **datovou strukturu** nebo funkci **%fields** u UPDATE.
- Soubor se otevře vždy při vstupu do podprocedury. Uzavírá se při výstupu z podprocedury a proměnné zanikají.
- Klíčové slovo **STATIC** u souboru nechá soubor otevřený a zachová jeho data pro příští volání.

Program s voláním funkce

Program VZUPKA/FU_CEN_C

```
**free
```

```
Dcl-DS cenik_ds extname('CENIKP') qualified End-DS;
```

```
Dcl-S  cena_materialu Like(cenik_ds.CENAJ) Inz;
```

```
Dcl-PI *N;
```

```
    material Like(cenik_ds.MATER);  // vstupní parametr
```

```
End-PI;
```

```
// volám SQL funkci
```

```
Exec SQL  set :cena_materialu
```

```
          = VRAT_CENU_MATERIALU (MATERIAL => :material );
```

```
Dsply ('cena_materialu: ' + %char(cena_materialu));
```

```
Return;
```

Vytvořím **modul** pro servisní program

```
CRTRPGMOD MODULE(FU_CEN_R) SRCFILE(QRPGLESRC)
```

Vytvořím **spojovací text** (binder source) v souboru QSRVSRC pro servisní program

```
STRPGMEXP SIGNATURE('VER1')  
EXPORT SYMBOL(FU_CEN_R)  
ENDPGMEXP
```

Vytvořím **servisní program**

```
CRTSRVPGM SRVPGM(FU_CEN_R) MODULE(*SRVPGM) SRCFILE(QSRVSRC) SRCMBR(*SRVPGM)  
.....
```

Vytvořím **modul** volajícího programu

```
CRTSQLRPGI OBJ(FU_CEN_C) SRCFILE(QRPGLESRC) SRCMBR(*OBJ) OBJTYPE(*MODULE)
```

Vytvořím **volající program** připojením servisního programu k modulu

```
CRTPGM PGM(FU_CEN_C) MODULE(*PGM) BNDSRVPGM((FU_CEN_R))  
.....
```

Spustím volající program testující UDTF funkci

```
CALL PGM(FU_CEN_C) PARM('00001')
```

User defined table function (UDTF) externí – vrací tabulku

Skript VZSQLPGM/OBJDAT_FE

Vytvoříme definiční **skript** pro externí uživatelskou funkci. Funkce OBJDAT_F přijímá dva parametry určující rozmezí datumů. Vybere objednávky z tabulky OBJHLA_T v daném rozmezí a **vytvoří tabulku** s čísly objednávek a s daty. Seznam bude uspořádán podle čísla objednávky vzestupně.

```
SET SCHEMA = VZSQL;  
CREATE OR REPLACE FUNCTION VZSQLPGM.OBJDAT_F (DATUM1 DATE, DATUM2 DATE)  
  RETURNS TABLE  
  ( COBJ CHAR(6) CCSID 870,  
    DTOBJ DATE )  
LANGUAGE RPGLE  
PARAMETER STYLE DB2SQL          -- umožňuje open, fetch, close  
NOT DETERMINISTIC  
READS SQL DATA  
SCRATCHPAD                      -- průběžná paměť  
PROGRAM TYPE SUB                -- funkce je ILE podprocedura  
NOT FENCED                      -- není vázána na stejnou úlohu  
NO FINAL CALL                   -- není první a poslední volání  
CARDINALITY 1000                -- přibližné omezení počtu výsledných řádků  
EXTERNAL NAME VZSQLPGM.OBJDAT_F(OBJDAT_F) -- serv. program a podprocedura
```

Skript umístíme do zdrojového členu, např. OBJDAT_FE a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(OBJDAT_FE) MARGINS(160)
```

UDTF externí – vytvoření funkce

VZSQLPGM/OBJDAT_F

Zdrojový **modul** OBJDAT_F s procedurou (funkcí) OBJDAT_F

```
**free
Ctl-Opt nomain;
Dcl-Proc OBJDAT_F Export;
  Dcl-DS OBJHLA_T Ext Template ; // host variables z tabulky OBJHLA_T
End-DS;
// Datová struktura dat přetrvávajících mezi voláními funkce
Dcl-DS ScratchDS Template;
  ScrLen Int(10:0) Inz(%Size(ScratchDS));
  Cntr Packed(6:0) Inz(0);
End-DS;
// Procedure interface
Dcl-PI *N; // parametry
  Datum1 Date; // in
  Datum2 Date; // in
  COBJ_PAR Like(COBJ); // out
  DTOBJ_PAR Like(DTOBJ); // out
  Datum1_ind Int(5:0); // in
  Datum2_ind Int(5:0); // in
  COBJ_PAR_ind Int(5:0); // out
  DTOBJ_PAR_ind Int(5:0); // out
  SQLSTATE_PAR Char(5); // out
  Func_name VarChar(517); // in
  Spec_name VarChar(128); // in
  Message_text VarChar(1000); // out
  Scratchpad LikeDS(ScratchDS); // inout
  CallType Int(10:0); // in
End-PI;
```

```

If Calltype = -1;          // OPEN
  Exec SQL declare CUR cursor for
    select COBJ, DTOBJ
    from OBJHLA_T
    where DTOBJ between :Datum1 and :Datum2
    order by COBJ;
  Exec SQL open CUR;
ElseIf Calltype = 0;      // FETCH
  // Aby kurzor přetrval jednotlivá volání, je nutné při kompilaci
  // zadat parametr CLOSQLCSR(*ENDACTGRP)
  Exec SQL fetch next from CUR into
    :COBJ_PAR :COBJ_PAR_ind,
    :DTOBJ_PAR :DTOBJ_PAR_ind ;
ElseIf Calltype = 1;      // CLOSE
  Exec SQL close CUR;
EndIf;

End-Proc OBJDAT_F;

```

Poznámka 1: Čítač *Scratchpad.Cntr* není použit, mohl by sloužit např. k omezení nebo tisku počtu vracených řádků.

Poznámka 2: Velmi důležitý je parametr CLOSQLCSR s hodnotou *ENDACTGRP zadaný při kompilaci modulu; zachovává otevřený kurzor mezi jednotlivými voláními procedury (Open, Fetch, Close). Kurzor se zavře až při ukončení aktivační skupiny. Předvolená hodnota *ENDMOD by způsobila, že kurzor by se zavřel po každém volání procedury (nejen při volání Close).

Poznámka 3: Jednotlivá volání jsou realizována jako vlákna (threads). V nich nejsou dostupné hodnoty proměnných pro výpis paměti (dump). Ladění je ale možné pomocí programu STRDBG.

Přijímá dva parametry a vrací tabulku obsahující čísla a data objednávek v rozmezí parametrů.

Program s voláním tabulkové funkce

Program VZSQLPGM/OBJDAT_P

```
**free
Dcl-PI *n;    // vstupní parametry
    Dcl-S Datum1A    Date;
    Dcl-S Datum2A    Date;
End-PI;

Dcl-S Datum1    Date; // parametry pro volanou funkci
Dcl-S Datum2    Date;

Dcl-S COBJ    Char(6); // host variables pro výsledky z funkce
Dcl-S DTOBJ    Date;

Datum1 = %date(DATUM1A);
Datum2 = %date(DATUM2A);

Exec SQL declare CUR cursor for
    select * from TABLE ( OBJDAT_F(DATUM1 => :Datum1 , DATUM2 => :Datum2 ) );

// Zpracování výsledků dotazu
Exec SQL open CUR ;
Exec SQL fetch CUR into :COBJ, :DTOBJ ;
DoW sqlstate = '00000';
    Snd-Msg COBJ + ' ' + %char(DTOBJ); // výpis do joblogu
    Exec SQL fetch CUR into :COBJ, :DTOBJ ;
EndDo;
Exec SQL close CUR;
Return;
```

Vytvořím **spojovací text** (binder source) v souboru QSRVSRC **pro servisní program** OBJDAT_F s exportem procedury OBJDAT_F

```
STRPGMEXP SIGNATURE( 'VER1' )  
EXPORT SYMBOL(OBJDAT_F)  
ENDPGMEXP
```

Vytvořím **servisní program** OBJDAT_F s procedurou (funkcí) OBJDAT_F

```
CRTSQLRPGI OBJ(VZSQLPGM/OBJDAT_F) SRCFILE(QRPGLESRC) SRCMBR(OBJDAT_F)  
OBJTYPE(*SRVPGM) CLOSQLCSR(*ENDACTGRP)
```

Vytvořím **modul** pro volající program OBJDAT_P

```
CRTSQLRPGI OBJ(VZSQLPGM/OBJDAT_P) SRCFILE(QRPGLESRC) SRCMBR(OBJDAT_P)  
OBJTYPE(*MODULE)
```

Vytvořím **program** OBJDAT_P s připojeným servisním programem OBJDAT_F

```
CRTPGM PGM(VZSQLPGM/OBJDAT_P) MODULE(*PGM) BNDSRVPGM((VZSQLPGM/OBJDAT_F))
```

Volám program OBJDAT_P z příkazového řádku

```
CALL VZSQLPGM/OBJDAT_P ( '2012-01-01' '2026-12-31' )
```

nebo CL příkazem

```
CMD          PROMPT('Volání SQL programu OBJDAT_P')
PARM         KWD(DATUM1) TYPE(*CHAR) LEN(10) +
              DFT('2013-01-01') +
              PROMPT('Datum OD')
PARM         KWD(DATUM2) TYPE(*CHAR) LEN(10) +
              DFT('2022-12-31') +
              PROMPT('Datum OD')
```

Výsledek v job logu

```
3 > OBJDAT_P DATUM1('2012-01-01') DATUM2('2026-12-31')
000001 2025-04-05
000002 2022-12-14
000003 2022-12-14
```

Příklady

SQL tabulky, pohledy, procedury nebo tabulkové funkce v knihovnách **QSYS2** a **SYSTOOLS** nahrazují systémové funkce API. Snadno se používají v prostředí ACS. Lze je použít i v RPG. Jejich přehled je v dokumentaci <https://www.ibm.com/docs/en/i/7.5.0?topic=optimization-i-services>.

Funkce QSYS2.OBJECT_STATISTICS – zjišťování údajů o objektech

Tabulková funkce **OBJECT_STATISTICS** funguje podobně jako CL příkazy DSPOBJD, RTVOBJD nebo API QUSROBJD. Parametry mají jména, která můžeme použít s oddělovačem =>. Nemusíme je používat vůbec. Popis funkce je [zde](#).

Volání tabulkové funkce ve skriptu ACS nebo STRSQL

```
SELECT objname, objtype, objcreated, objsize
FROM TABLE ( QSYS2.OBJECT_STATISTICS
              (object_schema => 'VZUPKA',
               objtypelist => 'PGM, MODULE, SRVPGM',
               object_name => 'FU_CEN_*') )
```

Výsledek v ACS:

OBJNAME	OBJTYPE	OBJCREATED	OBJSIZE
FU_CEN_C	*PGM	2025-03-29 11:30:31.000000	176128
FU_CEN_C	*MODULE	2025-03-29 11:21:46.000000	135168
FU_CEN_R	*MODULE	2025-03-29 11:30:12.000000	163840
FU_CEN_R	*SRVPGM	2025-03-29 11:30:25.000000	233472

Volání tabulkové funkce QSYS2.OBJECT_STATISTICS a procedury QSYS2.QCMDEXC

Program OBJSTAT3

Tiskne výsledky a tiskový soubor převádí CL příkazem CPYSPLF (pomocí QSYS2) do IFS.

```
dcl-f QSYSPRT printer(120);
dcl-s objname      varchar(10);
dcl-s objtype      varchar(8) ;
dcl-s objcreated   timestamp;
dcl-s objsize      packed(15);
dcl-s col_names    varchar(120);
dcl-s columns      varchar(120);
dcl-s cmdText      char(200);
if not %open(QSYSPRT);
    open QSYSPRT; // otevřu tiskový soubor
endif;
col_names = 'OBJNAME,OBJTYPE,OBJCREATED,OBJSIZE'; // jména sloupců
except header; // tisk jmen sloupců
Exec SQL declare CS cursor for
    SELECT objname, objtype, objcreated, objsize
        FROM TABLE ( QSYS2.OBJECT_STATISTICS // tabulková funkce
            ('VZUPKA','PGM, MODULE, SRVPGM', 'FU_CEN_*') );
Exec SQL open CS;
Exec SQL fetch from CS
    into :objname, :objtype, :objcreated, :objsize ;
dow sqlstate < '02000';
    columns = '' + objname + ''',' + objtype + ''',' +
        %char(objcreated) + ',' + %char(objsize); // hodnoty sloupců
    except detail; // tisk řádku s hodnotami sloupců
    Exec SQL fetch from CS
        into :objname, :objtype, :objcreated, :objsize ;
enddo;
Exec SQL close CS;
```

```

close QSYSPRT; // zavřu tiskový soubor
cmdText = 'CPYSPLF FILE(QSYSPRT) TOFILE(*TOSTMF) SPLNBR(*LAST) +
          TOSTMF(''/home/vzupka/objstat3.txt'') STMFOPT(*REPLACE)';
Exec SQL CALL QSYS2.QCMDEXC(:cmdText); // volání SQL procedury
return;

```

```

OQSYSPRT    E          header
O           col_names
OQSYSPRT    E          detail
O           columns

```

Tiskový výstup a obsah souboru /home/vzupka/objstat3.txt

```

OBJNAME,OBJTYPE,OBJCREATED,OBJSIZE
'FU_CEN_C','*PGM',2025-03-29-11.30.31.000000,176128
'FU_CEN_C','*MODULE',2025-03-29-11.21.46.000000,135168
'FU_CEN_R','*MODULE',2025-03-29-11.30.12.000000,163840
'FU_CEN_R','*SRVPGM',2025-03-29-11.30.25.000000,233472

```

Zobrazení v tabulkovém editoru po přejmenování na objstat3.csv

OBJNAME	OBJTYPE	OBJCREATED	OBJSIZE
'FU_CEN_C'	'*PGM'	2025-12-23-18.38.50.000000	176128
'FU_CEN_C'	'*MODULE'	2025-03-29-11.21.46.000000	135168
'FU_CEN_R'	'*MODULE'	2025-03-29-11.30.12.000000	163840
'FU_CEN_R'	'*SRVPGM'	2025-03-29-11.30.25.000000	233472

Funkce QSYS2.SYSSCHEMAS a procedura QSYS2.IFS_WRITE

Program SYSSCH_IFS

Popis funkce QSYS2.SYSSCHEMAS je v publikaci

<https://www.ibm.com/docs/en/i/7.5.0?topic=reference-db2-i-catalog-views> a konkrétně [zde](#).

Popis funkce QSYS2.IFS_WRITE je v publikaci

<https://www.ibm.com/docs/en/i/7.5.0?topic=optimization-i-services> speciálně [zde](#).

Sloupec `SCHEMA_TEXT` je ve výsledné tabulce definován jako `VARGRAPHIC(50) CCSID 1200 Nullable`, proto je uveden příkaz `ctl-opt CCSID(*GRAPH:*IGNORE)`; a k němu jeho null-indikátor `SCHEMA_IND`.

```
**free
ctl-opt CCSID(*GRAPH:*IGNORE);
dcl-s header varchar(200)
      inz('SCHEMA_NAME,SCHEMA_OWNER,SCHEMA_CREATOR,CREATION_DATE,+
SIZE,SCHEMA_TEXT,SYS_NAME,IASP,');
dcl-s line    varchar(200);
dcl-ds *n;
      schema_name    varchar (10);
      owner          varchar (10);
      creator        varchar (10);
      timestamp      timestamp(1);
      size           packed (9: 0);
      schema_text    varchar (30);
      schema_ind     int (5);
      sys_name       char (10);
      iasp           int (5);
end-ds;

dcl-s label2 varchar(30) inz(*all'-'); // nahrazuje hodnotu null
dcl-s iasp2 char(1);
```

Exec SQL

-- Create an empty IFS file

```
CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',  
                     LINE => '',  
                     FILE_CCSID => 1208,  
                     OVERWRITE => 'REPLACE',  
                     END_OF_LINE => 'NONE');
```

Exec SQL

-- Add the header to the IFS file

```
CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',  
                     LINE => :header,  
                     FILE_CCSID => 1208,  
                     OVERWRITE => 'APPEND',  
                     END_OF_LINE => 'CRLF') ;
```

Exec SQL declare CS cursor for

-- List of schemas

```
SELECT cast (SCHEMA_NAME as varchar (10)),  
       varchar (SCHEMA_OWNER, 10) owner,  
       cast (SCHEMA_CREATOR as varchar (10)),  
       CREATION_TIMESTAMP as timestamp,  
       SCHEMA_SIZE size,  
       varchar (SCHEMA_TEXT, 20) schema_text,  
       varchar (SYSTEM_SCHEMA_NAME, 10) sys_name,  
       cast (IASP_NUMBER as smallint) as iasp  
FROM QSYS2.SYSSCHEMAS  
WHERE substring(SCHEMA_NAME, 1, 1) <> 'Q'  
AND    substring(SCHEMA_NAME, 1, 3) <> 'SYS' ;
```

Exec SQL open CS;


```

// Write text lines to the IFS file
DoW *on;
  Exec SQL fetch from CS into :schema_name, :owner, :creator, :timestamp, :size,
                                : schema_text :schema_ind, :sys_name, :iasp ;

  if sqlstate = '02000';
    leave;
  endif;
  if schema_ind = -1; // je hodnota sloupce null?
    schema_text = schema_text 2;
  endif;
  iasp2 = %char(iasp); // 1 character only

  // construct text line
  line = %trim(schema_name) + ',' +
    %trim(owner) + ',' +
    %trim(creator) + ',' +
    %char(%date(timestamp)) + ',' +
    %trim(%editc(size: 'P')) + ',' +
    %trim(schema_text) + ',' +
    %trim(sys_name) + ',' +
    iasp2 + ',';

  Exec SQL
    -- Add the text line to the IFS file
    CALL QSYS2.IFS_WRITE(PATH_NAME => '/home/vzupka/schemas.csv',
                        LINE => :line,
                        FILE_CCSID => 1208,
                        OVERWRITE => 'APPEND',
                        END_OF_LINE => 'CRLF');

  enddo;
  Exec SQL close CS;
  return;

```

Obsah souboru /home/vzupka/schemas.csv

```

SCHEMA_NAME,SCHEMA_OWNER,SCHEMA_CREATOR,CREATION_DATE,SIZE,SCHEMA_TEXT,SYS_NAME,IASP,
#COBLIB,QSYS,QLPINSTALL,2024-04-24,155648,-----,#COBLIB,0,
#LIBRARY,QSYS,QLPINSTALL,2024-04-24,114688,-----,#LIBRARY,0,
#RPGLIB,QSYS,QLPINSTALL,2024-04-24,139264,-----,#RPGLIB,0,
CORPDATA,VZUPKA,VZUPKA,2014-02-21,147456,COLLECTION - created,CORPDATA,0,
ILEDITOR,VZUPKA,VZUPKA,2025-09-04,114688,Code for i temporary,ILEDITOR,0,
KOLEKCE,VZUPKA,VZUPKA,2025-10-02,196608,-----,KOLEKCE,0,
MFA_EMKA,VZUPKA,VZUPKA,2024-12-30,114688,-----,MFA_EMKA,0,
MTCLP,QSECOFR,QSECOFR,2025-11-23,131072,-----,MTCLP,0,
...

```

Zobrazení v tabulkovém editoru

SCHEMA_NAME	SCHEMA_OWNER	SCHEMA_CREATOR	CREATION_DATE	SIZE	SCHEMA_TEXT	SYS_NAME	IASP	
#COBLIB	QSYS	QLPINSTALL	2024-04-24	155648	-----	#COBLIB	0	
#LIBRARY	QSYS	QLPINSTALL	2024-04-24	114688	-----	#LIBRARY	0	
#RPGLIB	QSYS	QLPINSTALL	2024-04-24	139264	-----	#RPGLIB	0	
CORPDATA	VZUPKA	VZUPKA	2014-02-21	147456	COLLECTION - created	CORPDATA	0	
ILEDITOR	VZUPKA	VZUPKA	2025-09-04	114688	Code for i temporary	ILEDITOR	0	
KOLEKCE	VZUPKA	VZUPKA	2025-10-02	196608	-----	KOLEKCE	0	
MFA_EMKA	VZUPKA	VZUPKA	2024-12-30	114688	-----	MFA_EMKA	0	
MTCLP	QSECOFR	QSECOFR	2025-11-23	131072	-----	MTCLP	0	

...

Dodatky

Trigger interní – výkonný skript

Skript TG_CENA_U

```
-- Trigger BEFORE UPDATE pro tabulku CENIKP
CREATE OR REPLACE TRIGGER TG_CENA_U BEFORE UPDATE ON CENIKP
  REFERENCING OLD ROW AS old_row
                NEW ROW AS new_row
  FOR EACH ROW                                -- spustí se u každého řádku
  MODE DB2ROW                                  -- předepsáno pro BEFORE
  WHEN (new_row.CENAJ > 100 and
        new_row.CENAJ > old_row.CENAJ * 1.10 )
  BEGIN
    SET new_row.CENAJ = old_row.CENAJ * 1.10;
    SIGNAL SQLSTATE VALUE '01H01' SET MESSAGE_TEXT =
      'Navýšení množství přesahuje 10 %. Ponechá se navýšení 10 %';
  END ;
```

Umístíme skript do zdrojového členu TG_CENA_U a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(TG_CENA_U) MARGINS(160)
```

Smazání

```
DROP TRIGGER TG_CENA_U
```

```
RMVPFTRG FILE(VZUPKA/CENIKP) TRG(TG_CENA_U)
```

Procedura interní – výkonný skript

Skript PR_CEN (v jazyku PL/SQL)

```
set schema = VZRPGE_FREE;  
create or replace procedure VZSQLPGM.CENA_MAT  
  ( in CISLO_ZAVODU   CHAR(2),          -- číslo závodu  
    in CISLO_SKLADU   CHAR(2),          -- číslo skladu  
    out CASTKA_CELKEM decimal (11, 2)   -- Součet částek cena krát množství  
  )  
language SQL  
reads sql data  
begin  
  -- zjištění ceny materiálů v daném závodu a skladu  
  select sum(CENA * MNOZ) into CASTKA_CELKEM  
    from STAVYP as S  
   join CENIKP as C on C.MATER = S.MATER  
  where SKLAD = CISLO_SKLADU and ZAVOD = CISLO_ZAVODU ;  
end
```

Umístíme skript do zdrojového členu PR_CEN a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(PR_CEN) MARGINS(160)
```

nebo

v aplikaci **IBM i Access Client Solutions** vložíme skript do okna **Run SQL Scripts** a spustíme.

UDTF interní – výkonný skript

PL/SQL skript OBJDAT_F

Skript generuje uživatelskou funkci OBJDAT_F, která přijímá dva parametry a **vrátí tabulku** se dvěma sloupci, COBJ (číslo objednávky) a DTOBJ (datum objednávky). Objednávky vybere z tabulky OBJHLA_T v daném rozmezí.

```
SET SCHEMA = VZSQL;  
CREATE OR REPLACE FUNCTION VZSQLPGM.OBJDAT_F ( DATUM1 DATE, DATUM2 DATE )  
RETURNS TABLE  
    ( COBJ CHAR(6) CCSID 870,  
      DTOBJ DATE )  
LANGUAGE SQL  
BEGIN  
    RETURN select COBJ, DTOBJ from OBJHLA_T  
           where DTOBJ between DATUM1 and DATUM2  
           order by COBJ;  
END
```

Skript umístíme do zdrojového členu, např. OBJDAT_F a spustíme CL příkazem

```
RUNSQLSTM SRCFILE(QSQLSRC) SRCMBR(OBJDAT_F) MARGINS(160)
```

Výsledkem skriptu je **servisní program OBJDA00001** v zadané knihovně. Ten je nutné spojit s modulem OBJDAT_P, aby vznikl **program OBJDAT_P**:

```
CRTPGM PGM(VZSQLPGM/OBJDAT_P) MODULE(*PGM) BNDSRVPGM((VZSQLPGM/OBJDA00001))
```

Procedurální jazyk SQL PL

Programovací jazyk SQL je popsán v dokumentaci

<https://www.ibm.com/docs/en/i/7.5.0?topic=reference-sql-procedural-language-sql-pl>,

<https://www.ibm.com/docs/en/i/7.5.0?topic=pl-sql-procedure-statement>.

Obširný výklad: <https://www.redbooks.ibm.com/redbooks/pdfs/sg248326.pdf>.

Řídící SQL příkazy

<i>label</i> :	návěští u příkazu
<i>GOTO label</i>	skok na návěští
<i>BEGIN .. END</i>	složený příkaz
<i>SET proměnná = výraz</i>	přiřazovací příkaz
<i>PIPE (výraz, výraz, ...)</i>	vytvoření záznamu v tabulkové funkci
<i>CALL procedura (parametry)</i>	volání procedury
<i>IF podmínka .. ELSEIF .. ENDIF .. END IF</i>	podmiňovací příkaz
<i>CASE [výraz] WHEN podmínka .. WHEN podmínka ..</i> <i>ELSE .. END CASE</i>	výběr alternativ
<i>FOR kurzor FOR select DO .. END FOR</i>	cyklus přes kurzor
<i>LOOP .. END LOOP</i>	neomezený cyklus
<i>WHILE podmínka DO .. END WHILE</i>	cyklus dokud ano
<i>REPEAT .. UNTIL podmínka END REPEAT</i>	cyklus dokud ne
<i>ITERATE label</i>	nové opakování cyklu s návěštím
<i>LEAVE label</i>	opuštění cyklu nebo bloku s návěštím
<i>RETURN výraz</i>	návrat z rutiny
<i>SIGNAL</i>	zaslání zprávy
<i>RESIGNAL</i>	zaslání zprávy do vyšší úrovně
<i>GET DIAGNOSTICS</i>	získání informace o stavu výpočtu

Složený příkaz

Příkazy uvnitř složeného příkazu musí být ukončeny středníkem.

BEGIN

DECLARE *variable*|*condition*|*retcode*

DECLARE *cursor*|*handler*

řídící SQL příkazy (včetně složeného)

+ *další SQL příkazy:*

SELECT INTO, OPEN, FETCH, PIPE, CLOSE, INSERT, UPDATE, DELETE,

CREATE, ALTER, DROP, ...

END

Obecný tvar triggeru

CREATE OR REPLACE TRIGGER *jméno*

BEFORE | AFTER | INSTEAD OF

INSERT | DELETE | UPDATE

ON *table*|*view*

REFERENCING NEW|OLD AS *zkratka*

FOR EACH STATEMENT|ROW MODE DB2SQL|DB2ROW

volby NOT SECURED|SECURED ENABLE|DISABLE ...

SET OPTIONS *volby*

WHEN (*podmínka*)

jediný-příkaz – jednoduchý nebo složený

Obecný tvar procedury

```
CREATE OR REPLACE PROCEDURE jméno
  ( parametry in | out | inout )
volby LANGUAGE SQL MODIFIES|READS SQL DATA ...
SET OPTIONS volby
tělo-rutiny – jednoduchý nebo složený příkaz
```

Obecný tvar uživatelské funkce UDF

```
CREATE OR REPLACE FUNCTION jméno
  ( vstupní parametry )
RETURNS výraz
volby LANGUAGE SQL MODIFIES|READS SQL DATA ...
SET OPTIONS volby
tělo-rutiny – jednoduchý nebo složený příkaz, musí obsahovat příkaz RETURN
```

Obecný tvar uživatelské tabulkové funkce UDTF

```
CREATE OR REPLACE FUNCTION jméno
  ( vstupní parametry )
RETURNS TABLE ( definice-sloupců )
MODIFIES|READS SQL DATA
volby LANGUAGE SQL MODIFIES|READS SQL DATA ...
SET OPTIONS volby
tělo-rutiny
  jednoduchý nebo složený příkaz,
  s příkazem PIPE musí obsahovat aspoň jeden příkaz RETURN bez argumentu,
  bez příkazu PIPE přesně jeden příkaz RETURN vracející tabulku
```


Příklad tabulkové funkce – hodnoty ve skladech

Program VZUPKA/PIPE

-- Funkce vrátí tabulku závodů a skladů s hodnotou materiálu podle velikosti

```
CREATE OR REPLACE FUNCTION HODNOTA()  
  RETURNS TABLE ( zavod CHAR(2), sklad CHAR(2), hodnota DECIMAL(11, 2) )  
  BEGIN  
    FOR cur CURSOR FOR  
      SELECT zavod, sklad, SUM(cenaj * mnoz) hodnota  
      FROM stavyp s JOIN cenikp c on s.mater = c.mater  
      GROUP BY zavod, sklad  
      ORDER BY hodnota DESC  
    DO  
      PIPE (zavod, sklad, hodnota); -- vytvoří další záznam  
    END FOR;  
    RETURN; -- povinný pro PIPE na konci výpočtu  
  END
```

Vyvolání STRSQL:

```
SELECT * FROM TABLE ( HODNOTA() )
```

Výsledek:

ZAVOD	SKLAD	HODNOTA
01	01	60,158.16
01	02	19,793.75
02	02	7,930.00
02	01	3,661.10
01	03	1.25

Dynamický SELECT s neznámým seznamem sloupců

Program DYNSEL_N

Kódy pro typy dat lze nalézt v dokumentaci <https://www.ibm.com/docs/en/i/7.5.0?topic=reference-sql-da-sql-descriptor-area> a speciálně [zde](#).

```
Ctl-Opt dftactgrp(*no) actgrp(*new);

Dcl-F QPRINT PRINTER(200) OFLIND(*INOA);

// Pomocné proměnné
Dcl-S DataSpace Char(1000);
Dcl-S Data Char(1000) Based(Ptr);
Dcl-S Ptr Pointer;
Dcl-S SQL_STATEMENT Char(500);
Dcl-S Idx Int(10:0);
Dcl-C MaxPocet Const(20);
Dcl-S JmenoSloupce Char(12);
Dcl-S Hodnota VarChar(100);
Dcl-S Digits Int(10:0);
Dcl-S DecPos Int(10:0);
Dcl-S Size Int(10:0);

// SQL Descriptor Area
Dcl-DS SQLDA ALIGN QUALIFIED INZ;
    SQLDAID Char(8);
    SQLDABC Int(10:0);
    SQLN Int(5:0) Inz(MaxPocet);
    SQLD Int(5:0);
    SQL_VAR DIM(MaxPocet) LIKEDS(SQLVAR);
End-DS;
```

```

// Šablona datové struktury SQLVAR pro proměnné
Dcl-DS SQLVAR                                TEMPLATE;
    SQLTYPE                                Int(5:0);
    SQLLEN                                Int(5:0);
    SQLRES                                Char(12);
    SQLDATA                                Pointer;
    SQLIND                                Pointer;
    SQLNAME                                VarChar(30);
End-DS;
Dcl-DS *N;  // Překrytí pakovaného čísla znaky
    Packed                                Packed(39:9) INZ;
    PackedChar                            Char(20) OVERLAY(Packed);
End-DS;
SQL_STATEMENT = 'select STAVYP.MATER, CENAJ, NAZEV +
                from CENIKP, STAVYP +
                where (CENAJ * MNOZ) between 0.00 and 99.00 +
                and CENIKP.MATER = STAVYP.MATER +
                order by STAVYP.MATER asc +
                for read only';
Exec SQL prepare SEL from :SQL_STATEMENT ;
Exec SQL describe SEL into :SQLDA;
Exec SQL declare CUR cursor for SEL;
Exec SQL open CUR ;

Exsr Adresy_SQLDA;  // dosadit adresy proměnných do SQLDA

// přečíst a zpracovat data z dotazu
Exec SQL fetch next from CUR using descriptor :SQLDA ;
DoW SQLSTATE < '02000';
    Exsr ZpracData;  // zpracovat data z proměnných
    Exec SQL fetch next from CUR using descriptor :SQLDA ;
EndDo;
return;

```

```

BegSr Adresy_SQLDA; // dosazení adres proměnných do SQLDA
  Ptr = %addr(DataSpace); // ukazatel na přijímaná data
  For Idx = 1 to SQLDA.SQLD; // skutečný počet sloupců
    SQLDA.SQL_VAR(Idx).SQLDATA = Ptr ; // ukazatel na položku dat
    Ptr += %size(Hodnota); // stejná délka pro všechny typy dat
  EndFor;
Endsr;

BegSr ZpracData;
  Ptr = %addr(DataSpace);
  For Idx = 1 to SQLDA.SQLD; // skutečný počet sloupců
    JmenoSloupce = SQLDA.SQL_VAR(Idx).SQLNAME;
    // typ CHAR (452) nebo DATE (384) se přesune přímo
    If SQLDA.SQL_VAR(Idx).SQLTYPE = 452 or
      SQLDA.SQL_VAR(Idx).SQLTYPE = 384;
      Hodnota = Data;
      // typ DECIMAL je třeba převést do znakové podoby
    ElseIf SQLDA.SQL_VAR(Idx).SQLTYPE = 484;
      Digits = %div(SQLDA.SQL_VAR(Idx).SQLLEN : 256); // horní bajt
      DecPos = %rem(SQLDA.SQL_VAR(Idx).SQLLEN : 256); // spodní bajt
      Size = %div(Digits: 2) + 1; // počet bajtů pakovaného čísla
      Packed = 0; // inicializovat pomocnou proměnnou
      PackedChar = %replace( %subst(Data: 1: Size): PackedChar:
        %size(PackedChar) - Size + 1: %size(PackedChar));
      Eval(H) Packed = Packed * 10 ** (%decpos(Packed) - Decpos);
      Hodnota = %editc(Packed: 'P');
    EndIf;

    Except DETAIL;
      Ptr += %size(Hodnota);
    EndFor;
  Except ODDELOVAC;
Endsr;

```

OQPRINT	E	DETAIL	1	
O		JmenoSloupce		
O		Hodnota		+2
OQPRINT	E	ODDELOVAC	1	
O				' _ _ _ _ _ _ _ _ _ _ '

Výsledky:

MATER	00002	
CENAJ		10,500000000
NAZEV	Zubní pasta Kalodont	

MATER	00003	
CENAJ		6,750000000
NAZEV	Prádelní šňůra	

MATER	00006	
CENAJ		16,050000000
NAZEV	Ponožky pánské bílé, nové	

Data pro příklady

Program INS_CEN

```
**free
Exec SQL SET OPTION COMMIT = *NONE ;
Exec SQL DELETE FROM CENIKP;
Exec SQL INSERT INTO CENIKP values ('00002', 459.00, 'Zubní pasta Kalodont');
Exec SQL INSERT INTO CENIKP values ('00001', 8.99, 'PIŠKOTY OPAVIA');
Exec SQL INSERT INTO CENIKP values ('00003', 1.25, 'Prádelní šňůra');
Exec SQL INSERT INTO CENIKP values ('00004', 10.50, 'Ponožky pánské tmavé');
Exec SQL INSERT INTO CENIKP values ('00005', 120.00, 'Tričko bílé');
Exec SQL INSERT INTO CENIKP values ('00006', 10.55, 'Ponožky pánské bílé, nové');
Exec SQL INSERT INTO CENIKP values ('00007', 1510.00, 'Kalhoty manšestrové');
Exec SQL INSERT INTO CENIKP values ('00008', 1700.00, 'Kalhoty džínové');
Exec SQL INSERT INTO CENIKP values ('00009', 250.00, 'Whisky Balantine');
Exec SQL INSERT INTO CENIKP values ('00010', 6500.00, 'Koňak Gruzínský');
Exec SQL INSERT INTO CENIKP values ('00011', 159.00, 'Taška sportovní');
Exec SQL INSERT INTO CENIKP values ('00012', 45.00, 'Taška sportovní');
Exec SQL INSERT INTO CENIKP values ('00013', 360.00, 'Míč');
Exec SQL INSERT INTO CENIKP values ('00014', 3500.00, 'Sako tvídové, nadměr');
Exec SQL INSERT INTO CENIKP values ('00015', 12500.00, 'Motorové koště');
Exec SQL INSERT INTO CENIKP values ('00016', 380.00, 'Slepičí peří na polštáře');
Exec SQL INSERT INTO CENIKP values ('00017', 130.00, 'Hrábě dřevěné, kolíky plastové');
Exec SQL INSERT INTO CENIKP values ('00018', 56.00, 'Husí sádlo v konzervě');
return;
```

Program INS_STA

****free**

```
Exec SQL SET OPTION COMMIT = *NONE ;
Exec SQL DELETE FROM STAVYP;
Exec SQL INSERT INTO STAVYP values ('01', '01', '00001', 100.000);
Exec SQL INSERT INTO STAVYP values ('01', '01', '00002', 1.000);
Exec SQL INSERT INTO STAVYP values ('01', '01', '00003', 1.000);
Exec SQL INSERT INTO STAVYP values ('01', '02', '00003', 1.000);
Exec SQL INSERT INTO STAVYP values ('01', '02', '00009', 1.080);
Exec SQL INSERT INTO STAVYP values ('01', '02', '00010', 1.000);
Exec SQL INSERT INTO STAVYP values ('01', '03', '00003', 1.000);
Exec SQL INSERT INTO STAVYP values ('02', '01', '00005', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '01', '00006', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '01', '00008', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '01', '00019', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '02', '00009', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '02', '00011', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '02', '00014', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '02', '00018', 2.000);
Exec SQL INSERT INTO STAVYP values ('02', '02', '00019', 2.000);
return;
```

Program INS_OBR

****free**

```
Exec SQL SET OPTION COMMIT = *NONE ;
Exec SQL DELETE FROM OBRATP;
Exec SQL INSERT INTO OBRATP values ('01', '01', '00001', 10.00 );
Exec SQL INSERT INTO OBRATP values ('01', '01', '00001', -5.00 );
Exec SQL INSERT INTO OBRATP values ('01', '02', '00002', 9.00 );
Exec SQL INSERT INTO OBRATP values ('01', '02', '00002', 1.00 );
Exec SQL INSERT INTO OBRATP values ('02', '01', '00003', 1.00 );
Exec SQL INSERT INTO OBRATP values ('02', '02', '00004', 1.00 );
return;
```